

Firefly

Day 2 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“choose a single height”*
- ▶ *“minimum number of obstacles the firefly needs to destroy”*

Programming interpretation:

- ▶ Each possible height can be evaluated independently.
- ▶ Bottom and top obstacles use different hit thresholds.
- ▶ Sorting both obstacle lists lets each threshold count be answered by binary search.

Textbook connection: CP4 3.3.1 Binary Search

Core technique: Sorted arrays plus binary search

- ▶ Obstacles alternate between bottom stalagmites and top stalactites.
- ▶ For every possible flying height, count how many obstacles are hit.
- ▶ Return the minimum hit count and how many heights achieve it.

- ▶ For a fixed height, an obstacle hits if its length is at least a threshold.
- ▶ Counting values above a threshold in a sorted list is a binary search problem.
- ▶ Bottom and top obstacles have different threshold formulas.

- ▶ Split obstacle lengths into bottom and top lists by parity of input index.
- ▶ Sort both lists.
- ▶ For each height h , bottom hits are lengths $\geq h$.
- ▶ Top hits are lengths $\geq H - h + 1$.
- ▶ Use `bisect_left` to count each group and update the minimum/count pair.

- ▶ `bisect_left(list, threshold)` returns the first index that can hit.
- ▶ `len(list) - index` is the number of hitting obstacles.
- ▶ Trying all heights is safe because each height query is logarithmic.

- ▶ Every possible height is evaluated exactly once.
- ▶ The binary searches compute exact hit counts because both lists are sorted.
- ▶ The maintained minimum and frequency therefore describe the best heights over the full range.

Complexity: $O(N \log N + H \log N)$ time and $O(N)$ memory.

- ▶ The top threshold uses $H - h + 1$ because heights are 1-indexed from the floor.
- ▶ Keep the two obstacle types separate; a single sorted list loses direction information.

Source lines: firefly.py:41-52

```
import sys
from bisect import bisect_left

def main():
    input = sys.stdin.buffer.readline

    n, h = map(int, input().split())

    bottom = []
    top = []
```

What this chunk does

- ▶ n is the number of obstacles and h is the cave height.
- ▶ Two lists separate obstacles that grow from the floor and hang from the ceiling.
- ▶ This separation matches the two different hit formulas.

Source lines: firefly.py:53-64

```
for i in range(n):
    length = int(input())

    if i % 2 == 0:
        bottom.append(length)
    else:
        top.append(length)

bottom.sort()
top.sort()
```

What this chunk does

- ▶ The problem states obstacles alternate bottom, top, bottom, top.
- ▶ Even positions enter bottom; odd positions enter top.
- ▶ Sorting is required before `bisect_left` can count obstacles by threshold.

Source lines: firefly.py:66-82

```
minimum = n
count = 0

for height in range(1, h + 1):

    index = bisect_left(bottom, height)
    bottom_hits = len(bottom) - index

    required_length = h - height + 1

    index = bisect_left(top, required_length)
    top_hits = len(top) - index

    hits = bottom_hits + top_hits
```

What this chunk does

- ▶ For a bottom obstacle, length at least height means it is hit.
- ▶ For a top obstacle, length at least $h - \text{height} + 1$ means it reaches down to that level.
- ▶ $\text{len}(\text{list}) - \text{index}$ gives the number of obstacles meeting the threshold.

Source lines: firefly.py:84-94

```
if hits < minimum:
    minimum = hits
    count = 1

elif hits == minimum:
    count += 1

print(minimum, count)

main()
```

What this chunk does

- ▶ A lower hit count resets both the minimum and the number of best heights.
- ▶ A tie increments the count of optimal heights.
- ▶ The final output is the minimum obstacles and the number of heights achieving it.

Solution file: `practice_contests/day_2/firefly.py`

Problem: <https://open.kattis.com/problems/firefly>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.